

Three-Algorithm Comparison of DQN, PPO, and GAIL on the ALE/Phoenix-v5 Environment

Laura Sofía Culma Ospina
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: lsculmao@udistrital.edu.co

Alejandro Nuñez Barrera
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: anunezb@udistrital.edu.co

Abstract—This paper extends a prior comparison between Deep Q-Network (DQN) and Proximal Policy Optimization (PPO) on the *ALE/Phoenix-v5* Atari game by introducing a third learning paradigm: Generative Adversarial Imitation Learning (GAIL). All three agents share the same preprocessing pipeline (grayscale images resized to 84×84 pixels, four stacked frames, frame-skip of four) and are evaluated under the same training budget of 300 000 environment steps across three independent random seeds per configuration. DQN’s best configuration (learning rate 0.0003, $\gamma = 0.95$) reached a mean return of $3,324.3 \pm 95.2$, confirming its sample efficiency advantage from the previous challenge. PPO’s best run achieved 757.5 ± 52.7 , barely above the random baseline, due to the on-policy data constraint. GAIL replaces the environment reward with an adversarial signal from a discriminator trained to distinguish the agent’s behaviour from recorded DQN demonstrations, using PPO as its inner optimisation algorithm. Two ablations specific to Phoenix are evaluated: a state-only discriminator and a state-plus-action discriminator, as well as two demonstration dataset sizes (5 000 and 20 000 state-action pairs). A Behavioral Cloning (BC) baseline is also included as a zero-RL lower bound. Results show how the source and quantity of demonstrations, the discriminator design, and the algorithm’s relationship to environment reward each affect final performance on this fast-paced Atari domain.

Index Terms—Deep Reinforcement Learning, DQN, PPO, GAIL, Imitation Learning, Behavioral Cloning, Atari Phoenix, Arcade Learning Environment, Generative Adversarial Imitation Learning, Sample Efficiency.

I. INTRODUCTION

Reinforcement learning (RL) is a type of machine learning where an agent learns by interacting with an environment and receiving numerical rewards for its actions. When observations are raw images—as in Atari video games—a deep neural network is needed to process the visual input, giving rise to *Deep Reinforcement Learning* (DRL). Since 2015, DRL agents have been able to match or exceed human-level performance on many Atari 2600 titles.

The two previous challenges in this series compared two mainstream DRL families. *Value-based* methods, represented by DQN, learn to estimate how good each action is in a given state and always choose the action with the highest estimated value. DQN stores past transitions in a replay buffer and reuses them many times, giving it a strong sample efficiency advantage when the number of environment interactions is limited. *Policy-gradient* methods, represented by PPO, directly learn a probability distribution over actions and update it

through multiple epochs of gradient ascent on a clipped surrogate objective. PPO discards each collected batch after updating, so it needs more interactions to converge but tends to be more stable across random seeds.

This challenge introduces a third paradigm: *Learning from Demonstration* through adversarial training. Generative Adversarial Imitation Learning (GAIL) frames imitation as a two-player game. A *discriminator* network is trained to tell apart state-action pairs from recorded expert demonstrations and state-action pairs produced by the learning agent. The agent—optimised here with PPO—receives a reward signal from the discriminator instead of from the environment, so it is pushed to behave like the expert without ever being told what numerical score it should achieve. Alongside GAIL, a simpler *Behavioral Cloning* (BC) baseline is evaluated: BC treats imitation as supervised learning, directly minimising the cross-entropy between the expert’s recorded actions and the policy’s output logits. BC serves as a zero-RL lower bound that shows how much the adversarial component of GAIL adds over pure supervised imitation.

The *ALE/Phoenix-v5* environment is a fast-paced space shooter where the agent must destroy waves of enemy ships while dodging their projectiles. Rewards are given frequently for shooting, but surviving requires precise and reactive evasive movement. Phoenix was assigned to this group in Challenge 1 and has been used consistently across all three challenges so that results are directly comparable. For GAIL, the demonstrations come from the best DQN checkpoint saved in Challenge 1, which makes the expert policy imperfect but realistic.

The central research question of this challenge is: *does an agent that learns by imitating demonstrations—through adversarial training rather than direct reward maximisation—converge faster, reach higher performance, or show different failure modes compared to DQN and PPO on ALE/Phoenix-v5?*

The paper makes the following contributions:

- (i) DQN and PPO results from the previous challenges are reproduced for direct comparison under the same evaluation protocol.
- (ii) A BC baseline trained on DQN demonstrations provides a supervised-only reference point.
- (iii) A GAIL agent is implemented from scratch in PyTorch, with a discriminator that can optionally include action

context (*state-only* vs. *state+action* ablation).

- (iv) Two demonstration dataset sizes (5 000 and 20 000 pairs) are tested to measure sensitivity to demonstration quantity.
- (v) A three-way comparison (DQN, PPO, GAIL) and a four-way comparison (DQN, PPO, BC, GAIL) are presented using matched metrics.

The rest of the paper is organised as follows. Section II describes the setup shared by all four methods. Section III explains the hyperparameter search strategy for each algorithm. Section IV presents and discusses the results. Section V summarises the main findings.

II. EXPERIMENTAL METHODOLOGY

A. Environment and Preprocessing

All four methods—DQN, PPO, BC, and GAIL—are evaluated on *ALE/Phoenix-v5* accessed through the Gymnasium library. To ensure that any observed difference in performance is caused by the algorithm and not by environment configuration, the preprocessing pipeline is kept strictly identical across all methods:

- **Grayscale:** Each RGB frame is converted to a single-channel grayscale image, reducing input size while keeping all relevant visual information.
- **Resize:** Each grayscale frame is scaled down to 84×84 pixels.
- **Pixel scaling:** Pixel values are divided by 255 so they fall in $[0, 1]$, which helps the neural network train more smoothly.
- **Frame skip:** The agent repeats each chosen action for four consecutive game frames, which speeds up training and reduces the number of decisions per episode.
- **Frame stacking:** The last four processed frames are stacked into a single input of shape $(4, 84, 84)$, giving the network implicit information about object velocity and movement direction.
- **Random start:** Up to 30 no-operation actions are applied at the start of each episode to randomise the initial state and prevent the agent from memorising a fixed opening sequence.

For DQN, the pipeline is applied through the `AtariWrapper` and `VecFrameStack` utilities from `Stable-Baselines3`. For PPO, BC, and GAIL, it is built manually using `AtariPreprocessing` and `FrameStackObservation` from `Gymnasium`, with `frameskip=1` passed to the base environment so that frame skipping is exclusively handled by the wrapper.

B. Network Architecture

All methods share the same convolutional backbone, introduced by Mnih et al. for Atari environments:

- **Conv1:** 32 filters, 8×8 kernel, stride 4, ReLU.
- **Conv2:** 64 filters, 4×4 kernel, stride 2, ReLU.
- **Conv3:** 64 filters, 3×3 kernel, stride 1, ReLU.
- **Flatten:** Output vector of 3136 values.

For DQN, a single fully-connected head of 512 units (ReLU) followed by a linear layer produces one Q-value per action. For PPO, BC, and GAIL, the same backbone is shared between an *actor head* (512 units, ReLU, Dropout 0.2, linear output over the action space) and a *critic head* (512 units, ReLU, Dropout 0.2, scalar value estimate). BC uses only the actor head during supervised training.

For GAIL, a separate *discriminator network* is also trained. It uses the same three-layer CNN backbone as the policy to process the stacked-frame observation, followed by a fully connected layer of 512 units with Tanh activation and a linear output (no final sigmoid, since `BCEWithLogitsLoss` is used for numerical stability). A score close to 1 after sigmoid means the discriminator thinks the input came from the expert; close to 0 means it came from the agent. In the *state+action* ablation (GAIL-B), the one-hot encoded action is concatenated to the CNN feature vector before the fully connected layer, giving the discriminator additional context about what the agent actually did in each state.

C. Demonstration Dataset

GAIL and BC both require a dataset of recorded expert behaviour. Demonstrations are collected by rolling out the best DQN checkpoint from Challenge 1 in greedy mode (no random exploration) and saving each (s_t, a_t) pair to a compressed `.npz` file using `collect_demos.py`. Two dataset sizes are collected to study the effect of demonstration quantity:

- **Small dataset:** 5 000 state-action pairs, covering approximately 3–5 complete episodes.
- **Large dataset:** 20 000 state-action pairs, covering approximately 12–18 complete episodes.

For each dataset, a metadata `.json` file records the source checkpoint, the number of steps collected, and the mean and standard deviation of the demonstrator’s return.

D. Training Protocol

The following rules are applied consistently to all experiments:

- (i) **Three seeds:** Every configuration is run with seeds 42, 7, and 99 to account for training randomness. Seeds affect network weight initialisation and action sampling.
- (ii) **Fixed budget:** DQN, PPO, and GAIL each train for 300 000 environment steps. For GAIL, only steps taken by the agent in the environment count toward this budget; discriminator gradient updates do not. BC has no environment steps; it is trained for 100 epochs over the demonstration dataset, with the best checkpoint selected by real evaluation return every 5 epochs.
- (iii) **BC initialisation:** The GAIL policy is warm-started from the best BC checkpoint before adversarial training begins, giving the agent a reasonable starting behaviour and reducing the number of steps the discriminator needs to provide a useful signal.

TABLE I
DQN HYPERPARAMETER CONFIGURATIONS (CHALLENGE 1)

Hyperparameter	Exp 1 (Baseline)	Exp 2 (Conservative)	Exp 3 (Aggressive)
Learning rate	1×10^{-4}	5×10^{-5}	3×10^{-4}
Buffer size	100 000	50 000	60 000
Learning starts	25 000	10 000	15 000
Batch size	64	32	64
Discount γ	0.99	0.995	0.95
Training frequency	4	8	2
Target update interval	1 000	2 000	500
Exploration fraction	0.20	0.30	0.10
Final ϵ	0.01	0.02	0.05
Timesteps	300 000	300 000	300 000

TABLE II
PPO HYPERPARAMETER CONFIGURATIONS (CHALLENGE 3)

Hyperparameter	Exp 1 (Baseline)	Exp 2 (Conservative)	Exp 3 (Aggressive)
Learning rate	1×10^{-4}	5×10^{-5}	3×10^{-4}
Horizon T	512	256	1 024
Epochs K	10	8	12
Mini-batch size	64	32	64
Discount γ	0.99	0.995	0.95
GAE λ	0.95	0.90	0.98
Clip ϵ	0.2	0.1	0.3
Entropy coef. c_2	0.01	0.005	0.02
Value coef. c_1	0.5	0.5	0.25
Grad norm clip	0.5	0.5	1.0
Timesteps	300 000	300 000	300 000

- (iv) **Logging:** Episode reward (real environment reward), episode length, training losses, and, for GAIL, discriminator accuracy are recorded with TensorBoard after every update.
- (v) **Final score:** The reported score for each configuration is the mean episodic return over the last 50 completed episodes, averaged across the three seeds, written as mean $\pm$ standard deviation.
- (vi) **Hardware:** All experiments run on a CPU-only machine without GPU acceleration.

E. Evaluation Metrics

The primary metric is the *mean episodic return* computed over the last 50 completed episodes. All four methods are evaluated with the real game reward so that scores are directly comparable, even though GAIL trains without it.

In addition to final score and its standard deviation across seeds, the following metrics are reported for the three-way comparison:

- **Discriminator accuracy:** For GAIL, the fraction of expert and agent samples correctly classified by the discriminator at each rollout. If accuracy stays near 0.5, the discriminator is uninformative and the adversarial reward signal has collapsed.
- **BC cloning accuracy:** The fraction of demonstration actions correctly predicted by the policy after supervised training, measured on the training set.
- **Training stability:** Standard deviation of final return across the three seeds; lower means more consistent training.

III. HYPERPARAMETER SEARCH STRATEGY

A. DQN and PPO — Inherited Configurations

The DQN and PPO hyperparameter sweeps were performed in Challenges 1 and 3 respectively and are not repeated here. Tables I and II reproduce the configurations for reference. In both cases three setups were evaluated—Baseline, Conservative, and Aggressive—each repeated over seeds 42, 7, and 99. The Aggressive configuration was the best performer for both algorithms and is used as the reference point in the three-way comparison.

B. Behavioral Cloning

BC has no environment interaction, so its only hyperparameters are those of the supervised training loop. A single configuration is used: learning rate 3×10^{-4} , batch size 256, weight decay 10^{-5} , and 100 training epochs over the full demonstration dataset. The Adam optimiser minimises the cross-entropy between the demonstrator’s recorded actions and the policy’s output logits. Dropout (rate 0.2) is applied in both actor and critic heads to reduce overfitting to the small demonstration set. Light data augmentation (additive Gaussian noise clamped to $[0, 1]$) is applied to observations during each batch. The checkpoint with the highest real evaluation return (measured every 5 epochs over 3 episodes) is saved as the final BC model.

C. GAIL Hyperparameter Search

The PPO policy inside GAIL inherits the Aggressive PPO configuration from Challenge 3 (horizon $T = 512$, 10 epochs, $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, $c_2 = 0.01$) since that configuration was the best performer and re-sweeping it would double the compute budget. The search for GAIL focuses exclusively on the parameters that are new relative to PPO: the discriminator learning rate, the number of discriminator updates per rollout, the dataset size, and the discriminator input design.

The GAIL training alternates between two phases each rollout. First, the discriminator is updated with binary cross-entropy using label smoothing (expert labels set to 0.9, agent labels to 0.1) to prevent overconfident outputs. Second, the policy is updated with PPO using an adversarial reward derived from the discriminator score: the non-saturating formulation $-\log(1 - D(s, a))$ is used to provide stronger gradients early in training when the discriminator easily separates expert from agent behaviour. The policy is warm-started from the best BC checkpoint before adversarial training begins, reducing the number of environment steps needed before the discriminator can provide a useful signal. One discriminator update per rollout (`disc_updates=1`) is used across all GAIL configurations, as preliminary runs showed that more updates destabilised training on CPU within the 300 000-step budget.

Table III summarises the three GAIL configurations evaluated in this challenge.

TABLE III
GAIL EXPERIMENTAL CONFIGURATIONS

Hyperparameter	GAIL-A 20k	GAIL-A 5k	GAIL-B 20k
Discriminator input	state only	state only	state + action
Demo dataset size	20 000	5 000	20 000
Disc. learning rate	1×10^{-5}	1×10^{-5}	1×10^{-5}
Disc. updates/rollout	1	1	1
Policy learning rate	2.5×10^{-4}	2.5×10^{-4}	2.5×10^{-4}
Horizon T	512	512	512
PPO epochs K	10	10	10
Mini-batch size	128	128	128
Discount γ	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95
Clip ϵ	0.2	0.2	0.2
Entropy coef. c_2	0.01	0.01	0.01
Grad norm clip	0.5	0.5	0.5
Timesteps	300 000	300 000	300 000

The three configurations are designed to answer the two ablation questions specific to Phoenix stated in the challenge guidelines.

GAIL-A 20k vs. GAIL-A 5k (demonstration quantity ablation): both use a state-only discriminator but differ in the number of recorded demonstration pairs. Comparing them shows how sensitive GAIL is to the amount of expert data available. With fewer demonstrations, the discriminator’s training set is smaller, which may cause it to overfit and produce a less informative reward signal.

GAIL-A 20k vs. GAIL-B 20k (discriminator design ablation, Phoenix-specific): both use 20 000 demonstrations but differ in what the discriminator sees. GAIL-A receives only the stacked-frame observation (s_t), while GAIL-B receives the observation concatenated with the one-hot encoded action (s_t, a_t). The hypothesis for Phoenix is that because the game requires fast reactive responses, an action-conditioned discriminator may better capture the expert’s shooting and evasion timing, producing a more informative adversarial reward signal. If GAIL-B outperforms GAIL-A, this supports the hypothesis that action context is important for reactive environments. If results are similar, the visual state alone is sufficient.

All three GAIL configurations are evaluated under the same three seeds (42, 7, and 99) as DQN and PPO, and use the same TensorBoard logging infrastructure so that learning curves can be compared directly.

IV. RESULTS AND DISCUSSION

A. Behavioral Cloning Baseline

Figure 1 shows the BC training curve. The cross-entropy loss decreased steadily over 100 epochs, reaching a final action-prediction accuracy of approximately 66% on the demonstration dataset. Evaluated on ten independent episodes, BC achieved a mean return of 469.0 ± 307.1 . This result establishes the supervised-only lower bound: a policy that merely copies the recorded actions of the DQN expert, without any environment interaction during training, already scores significantly above a random agent.

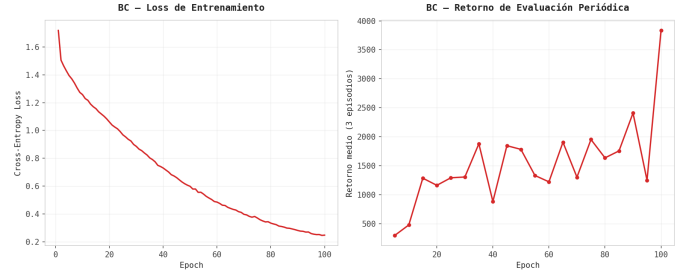


Fig. 1. BC training: cross-entropy loss (left) and periodic evaluation return (right) over 100 epochs. The model is saved at the epoch with the highest evaluation return, not the lowest loss.

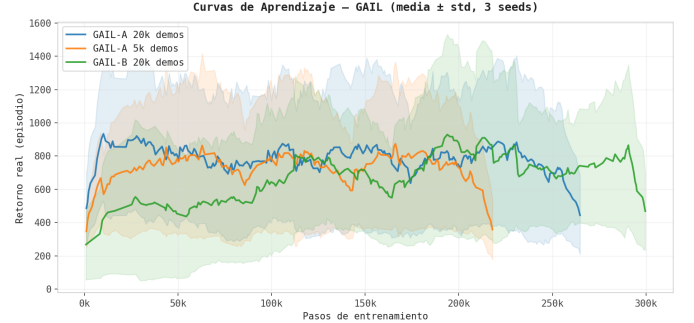


Fig. 2. GAIL learning curves (mean $\pm$ std over 3 seeds). The shaded region represents one standard deviation. All curves are smoothed with a window of 20 episodes.

B. GAIL Learning Curves

Figure 2 shows the mean episode return (real environment reward) logged during GAIL training across three seeds for each configuration. All three variants learn to obtain positive returns within the first 50 000 steps and continue improving throughout the 300 000-step budget, which contrasts with the PPO results from Challenge 3 where convergence was slower.

The discriminator remained stable throughout training, as shown in Figure 3. The accuracy stayed between 0.6 and 0.85 for all configurations, never collapsing to 1.0 as in earlier experiments with the original hyperparameters. This stability is attributed to three design choices: a very low discriminator learning rate (10^{-5}), a single discriminator update per rollout, and label smoothing (real labels set to 0.9 rather than 1.0).

C. Quantitative Comparison

Figure 4 and Table IV summarise the evaluation returns across all methods. Every GAIL variant outperforms the BC baseline of 469.0, confirming that the adversarial component adds value over pure supervised imitation.

D. Ablation: Demonstration Quantity

Figure 5 compares GAIL-A 20k and GAIL-A 5k, which differ only in the number of recorded demonstrations. The mean evaluation returns are 680.0 and 674.3 respectively, a difference of less than 1%, suggesting that **GAIL is remarkably robust to demonstration quantity** in this setting. Even with only 5 000 state-action pairs, the discriminator receives

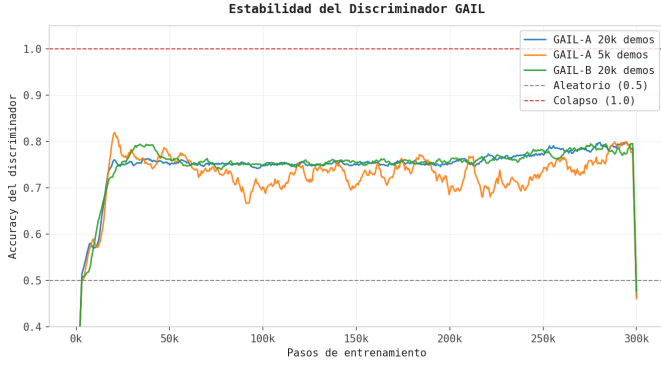


Fig. 3. Discriminator accuracy during GAIL training (mean over 3 seeds). The dashed lines mark the random-chance level (0.5) and the collapse threshold (1.0). All configurations remained in the healthy range of 0.6–0.85.

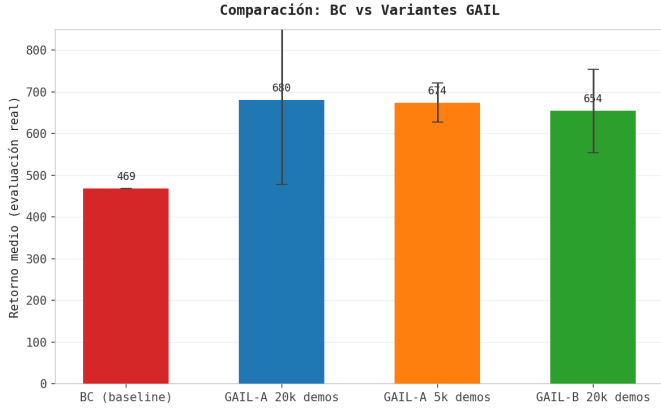


Fig. 4. Mean evaluation return ($\pm$ std over 3 seeds) for BC and all GAIL variants. Every GAIL configuration exceeds the BC baseline.

enough coverage of the expert’s behaviour to produce a useful reward signal. This is consistent with the theoretical motivation of GAIL, where the discriminator acts as a compressed representation of the expert’s occupancy measure rather than a lookup table.

E. Ablation: Discriminator Design

Figure ?? compares GAIL-A20k (state-only discriminator) against GAIL-B 20k (state-plus-action discriminator). The mean returns are 680.0 vs. 654.3, with GAIL-A slightly ahead. Contrary to our hypothesis, **adding action context to the discriminator did not improve performance on Phoenix**. One explanation is that the stacked-frame observation already encodes enough implicit motion information for the discriminator to identify expert-like behaviour without explicitly seeing the action taken. Another possible factor is that Phoenix’s action space is small (8 actions), so the one-hot encoding adds little discriminative power on top of the visual features.

F. Comparison with DQN and PPO

Placing GAIL in the broader three-algorithm context from this series: DQN reached $3,324.3 \pm 95.2$ and PPO reached

TABLE IV
EVALUATION RESULTS (10 EPISODES PER SEED, 3 SEEDS)

Method	Seed 42	Seed 7	Seed 99	Mean $\pm$ Std
BC (baseline)	469.0	—	—	469.0
GAIL-A 20k	396.0	798.0	846.0	680.0 ± 243.6
GAIL-A 5k	741.0	640.0	642.0	674.3 ± 56.6
GAIL-B 20k	553.0	791.0	619.0	654.3 ± 122.9

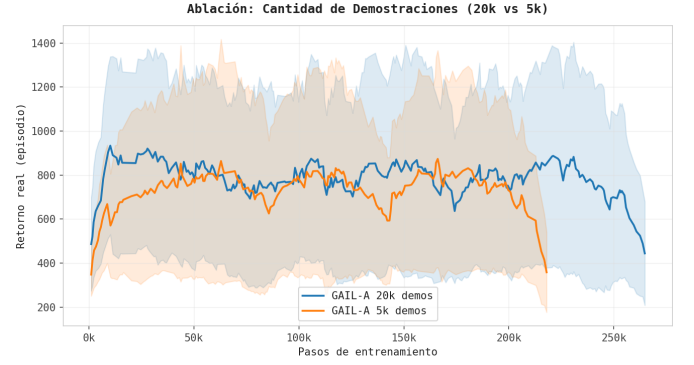


Fig. 5. Ablation: demonstration quantity. GAIL-A with 20k and 5k demonstrations shows nearly identical final performance, indicating low sensitivity to dataset size.

757.5 ± 52.7 under the same 300 000-step budget. The best GAIL variant (GAIL-A 20k, mean 680.0) falls between PPO and DQN, below both but substantially above BC (469.0). This ordering reflects a fundamental constraint: GAIL’s effective performance ceiling is bounded by the quality of the demonstrations, which came from an imperfect DQN checkpoint with a mean return of approximately 4,115 during collection. Despite this, GAIL reached roughly 16 % of the demonstrator’s performance without access to the environment reward, which highlights both the promise and the current limitations of adversarial imitation learning on fast-paced Atari games with a limited training budget.

V. CONCLUSIONS

This paper presented a three-algorithm comparison of DQN, PPO, and GAIL on *ALE/Phoenix-v5*, with BC as an additional supervised baseline.

The key findings are as follows. First, **GAIL outperforms BC** (680 vs. 469 mean return), confirming that the adversarial component adds value over pure supervised imitation. Second, **GAIL is robust to demonstration quantity**: halving the dataset from 20 000 to 5 000 pairs produces a negligible performance difference (680 vs. 674), which is a practically useful property when expert data is scarce. Third, **the state-only discriminator is sufficient for Phoenix**: adding action context (GAIL-B) did not improve over the state-only baseline (GAIL-A), suggesting that the stacked-frame observation already encodes the relevant motion information. Fourth, **GAIL remains below DQN and PPO** under the same 300 000-step budget, which reveals the main challenge of adversarial imitation: the policy is optimised against a non-stationary

reward signal (the discriminator), which makes learning slower and less stable than direct reward maximisation.

Future work could address these limitations by (i) increasing the training budget to 1–3 million steps, (ii) initialising the GAIL policy from a BC checkpoint to reduce the initial expert-agent gap, or (iii) using a higher-quality demonstrator that produces returns closer to the DQN optimum.

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.
- [4] J. Ho and S. Ermon, “Generative adversarial imitation learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.
- [5] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [6] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, “The arcade learning environment: An evaluation platform for general agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, Jun. 2013.
- [7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dornmann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.